

Technical Report: Second-Look Lending

Break Through Tech AI Studio — American Express Challenge

Fall 2026

Abstract

This report presents a technical framework for the Second-Look Lending challenge, which asks participants to predict consumer-loan default risk and translate those predictions into profit-aware and inclusive lending decisions. The project focuses particularly on thin-file applicants, who have limited or no traditional credit-bureau history. The proposed approach combines relational data aggregation, temporal validation, probability modeling, calibration, and threshold-based decision optimization. Logistic regression provides an interpretable baseline, while random forests and gradient-boosted trees provide nonlinear alternatives. Model evaluation uses ROC-AUC, LogLoss, calibration and Brier score, while policy evaluation uses expected portfolio profit, a thin-file inclusion gap, and an Inclusive Profit Score.

1. Introduction

1.1 Problem Background

The Second-Look Lending challenge examines how machine-learning models can improve lending decisions for consumers who may not have enough traditional credit history for conventional credit-risk models. The challenge uses a synthetic Home Credit-style relational dataset and asks teams to identify applicants who are sufficiently creditworthy to approve while controlling portfolio risk and considering whether thin-file applicants are being excluded.

The central technical problem is therefore not simply binary classification. A useful system must produce reliable default probabilities and then convert those probabilities into lending decisions under an economic objective and a default-rate constraint.

1.2 Research Questions

1. How accurately can default risk be predicted using the available relational consumer-loan data?
2. How do logistic regression, random forests, and gradient-boosted trees compare in discrimination and calibration?
3. How should predicted probabilities be converted into approval decisions under the challenge's profit assumptions?
4. Does a single approval threshold disproportionately exclude thin-file applicants?
5. Can a decision policy improve thin-file inclusion without violating the portfolio default-rate constraint or substantially reducing expected profit?

2. Project Objectives

- Construct a leakage-resistant applicant-level feature table from the relational data.
- Establish a logistic-regression baseline.
- Train and compare random-forest and gradient-boosting models.
- Evaluate ROC-AUC, LogLoss, Brier score, and calibration.
- Use time-based validation reflecting the train/test chronology.
- Optimize approval decisions using the supplied economic assumptions.

- Measure the thin-file inclusion gap.
- Compare a naïve single-threshold policy against segment-aware alternatives.
- Produce a reproducible pipeline and basic Streamlit demonstration.

3. Dataset Characteristics and Relational Structure

The supplied synthetic dataset contains one million training loans and 200,000 test loans. Training observations cover WEEK_NUM values 0 through 91, while the test period covers weeks 92 through 107. The target variable is available only for training observations. The dataset is organized as multiple relational tables joined by case_id.

The data guide specifies that depth-0 tables contain one row per case_id, while depth-1 and depth-2 tables can contain multiple rows per case_id and therefore must be aggregated before being joined to the applicant-level base table. File suffixes such as _0 and _1 can represent sub-parts of the same logical table and should be concatenated.

3.1 Major Tables and Features

Table	Role	Example Variables
base	Applicant-level target and time	case_id, date_decision, MONTH, WEEK_NUM, target
static_0	Static applicant characteristics	income, credit amount, annuity, employment, education
static_cb_0	Credit assessment	risk assessment, number of queries
person_1	Person-level information	occupation, employment duration, housing type
credit_bureau_a_1	Credit-bureau history	credit amount, overdue amount, days past due
credit_bureau_a_2	Payment history	monthly payments, overdue payments
applprev_1	Previous applications	credit amount, approval date, status
tax_registry_a_1	Tax-registry information	amount, masked name/category

3.2 Thin-File Definition

The data guide identifies a built-in second-look signal: approximately 30% of applicants are thin-file because they have no rows in the credit_bureau_a tables. A practical implementation can define a thin-file indicator from the applicant's bureau row count. In the training data, thin-file applicants have a higher observed default rate than established-file applicants, while the documentation indicates that thin-file risk decreases over time.

4. Feature Engineering

Feature engineering should transform the relational tables into one row per case_id while preserving information about both the magnitude and structure of an applicant's history.

- Numeric amounts: mean, median, minimum, maximum, sum, standard deviation, and selected quantiles.
- Counts: number of bureau records, previous applications, payment records, and tax records.
- Recency: time since the most recent historical event when supported by available dates.
- Risk-history summaries: maximum and average days past due, overdue amounts, and payment behavior.

- Categorical variables: frequency counts, most common category, or one-hot encoding where appropriate.
- Missingness indicators where absence may contain useful information.
- Thin-file indicator: bureau row count equal to zero.

Care must be taken not to create features using information that would not have been available at the decision point. Aggregations should use only information available for each application, and future observations must not leak into the feature set.

5. Exploratory Data Analysis

- Measure overall default rate and compare thin-file with established-file rates.
- Examine how default rates change across WEEK_NUM.
- Compare feature distributions across default and non-default outcomes.
- Measure missingness and determine whether missingness is associated with the outcome.
- Inspect categorical cardinality and distributions.
- Examine correlations and redundancy among numeric variables.
- Compare thin-file and established-file populations across available applicant characteristics.
- Check for extreme or implausible values.

6. Data Preprocessing

The preprocessing pipeline should be fitted on training data only and then applied unchanged to validation and test data. Numeric features can be imputed and scaled for logistic regression, while tree-based models generally do not require scaling. Categorical variables should be encoded consistently across splits.

6.1 Leakage Prevention

- Do not use target-derived statistics as features.
- Fit imputers, scalers, and encoders only on the training fold.
- Avoid random splits when evaluating temporal generalization.
- Do not use solution.csv during model selection or feature engineering.
- Ensure historical aggregations exclude post-decision information.

7. Temporal Validation

Because the training data span weeks 0–91 and the test data span weeks 92–107, a temporal holdout is preferable to a purely random split. Earlier weeks can be used for model development while the latest training weeks serve as an out-of-time validation set.

This is especially important because the challenge documentation indicates temporal drift in thin-file default risk. An out-of-time validation set therefore provides a more realistic test of generalization to the later test period.

8. Candidate Models

8.1 Logistic Regression

Logistic regression is the primary interpretable baseline. For an applicant with feature vector X , the predicted probability of default is:

$$P(Y = 1 | X) = 1 / (1 + e^{-(\beta_0 + \beta^T X)})$$

Regularization should be tuned using the validation framework. Logistic regression provides a transparent reference point and helps determine whether more complex models provide meaningful gains.

8.2 Random Forest

Random forests provide a nonlinear alternative capable of modeling interactions without requiring explicit manual interaction terms. Key hyperparameters include number of trees, maximum depth, minimum samples per leaf, and feature-sampling strategy.

8.3 Gradient-Boosted Trees

Gradient boosting is a strong candidate for tabular credit-risk prediction because it can represent nonlinear relationships and high-order interactions. LightGBM, XGBoost, or another supported implementation can be evaluated using an out-of-time validation framework.

9. Model Evaluation

9.1 ROC-AUC

ROC-AUC measures ranking discrimination: the ability to assign higher predicted default risk to eventual defaulters than to non-defaulters. It is useful for comparing models but does not by itself establish probability calibration.

9.2 LogLoss

LogLoss evaluates the quality of predicted probabilities and strongly penalizes confident incorrect predictions. It is particularly relevant when probabilities are used to calculate expected profit.

$$\text{LogLoss} = -(1/N) \sum [y \log(p) + (1-y) \log(1-p)]$$

9.3 Brier Score and Calibration

The Brier score measures the mean squared difference between predicted probabilities and observed outcomes:

$$\text{Brier} = (1/N) \sum (p - y)^2$$

Reliability curves should compare predicted probabilities with observed default frequencies. If a model ranks applicants well but is poorly calibrated, post-hoc calibration such as Platt scaling or isotonic regression can be evaluated on held-out data.

10. Economic Decision Model

Under the supplied illustrative economics, approving a loan that is repaid produces +\$2,000, while approving a loan that defaults produces -\$8,000. A denied application produces \$0.

$$\text{Expected profit}(\text{approve}) = (1-p)(2000) + p(-8000) = 2000 - 10000p$$

The break-even point is $p = 0.20$. If no other constraint existed, approval would be economically favorable when predicted default probability is below 20%. The final policy must also satisfy the challenge's portfolio default-rate ceiling.

10.1 Threshold Optimization

Evaluate a grid of candidate thresholds on the validation set. For each threshold, calculate approval rate, realized default rate among approved applicants, expected profit, thin-file approval/inclusion, and the inclusion gap. Select the best policy subject to the required risk constraint.

11. Thin-File Inclusion and Fairness

A central challenge objective is preventing thin-file applicants from being systematically excluded because limited credit history is interpreted as high risk. The thin-file inclusion gap can be defined using approval among applicants who ultimately repay:

$$A_E = P(\text{Approve} = 1 \mid \text{Repay} = 1, \text{Established})$$

$$A_T = P(\text{Approve} = 1 \mid \text{Repay} = 1, \text{Thin-File})$$

$$\text{Inclusion Gap} = A_E - A_T$$

This metric captures whether qualified thin-file applicants are approved at a lower rate than qualified established-file applicants. It should be analyzed alongside overall profit and the default-rate constraint rather than optimized in isolation.

11.1 Inclusive Profit Score

The challenge's headline decision metric combines expected profit with an inclusion penalty:

$$\text{Inclusive Profit Score} = \text{Expected Profit} - \lambda \times \text{Inclusion Gap}$$

The penalty coefficient λ should be treated as a challenge-specified policy parameter. Sensitivity analysis across plausible values can show whether the selected policy is robust.

12. Policy Experiments

6. Naïve single-threshold policy: approve everyone below one global threshold.
7. Segment-aware threshold policy: use separate thin-file and established-file thresholds subject to the portfolio risk constraint.
8. Calibrated segment policy: calibrate probabilities by segment and apply a common economic threshold.
9. Profit-first policy: maximize expected profit subject to the default-rate ceiling.
10. Inclusive-profit policy: maximize Inclusive Profit Score subject to the default-rate ceiling.

These experiments should be compared on the same validation set and evaluated on both aggregate and segment-level outcomes.

13. Explainability and Responsible Deployment

Credit decisions require particular attention to explainability. The Consumer Financial Protection Bureau has stated that the use of complex or black-box algorithms does not remove obligations to provide specific and accurate reasons for adverse actions. A production-oriented system should therefore retain an explanation layer capable of translating model behavior into decision-relevant factors.

For this challenge, explainability can be investigated using logistic-regression coefficients, permutation importance, and tree-model feature importance or SHAP-style analyses. Explanations should be evaluated for stability and should not contradict the actual model.

14. Computational Architecture

The dataset is large enough that memory-aware processing is important. A practical architecture is to process each relational table separately, aggregate many-to-one tables by `case_id`, save compact feature tables, and then merge the resulting applicant-level tables.

- Data ingestion: pandas or another tabular processing tool.
- Feature construction: table-specific aggregations grouped by `case_id`.
- Modeling: scikit-learn plus LightGBM/XGBoost/CatBoost for boosted-tree experiments.
- Hyperparameter tuning: GridSearchCV or RandomizedSearchCV with temporal validation.
- Evaluation: ROC-AUC, LogLoss, Brier, calibration, expected profit, default rate, and inclusion gap.
- Presentation: Streamlit dashboard showing model metrics, threshold tradeoffs, and decision explanations.

15. Reproducible Machine-Learning Pipeline

11. Load and inspect the base table.
12. Load each related table and verify `case_id` multiplicity.
13. Concatenate fragments belonging to the same logical table.
14. Generate applicant-level aggregations.
15. Create thin-file and missingness indicators.
16. Merge feature tables with the base data.
17. Create an out-of-time validation split.
18. Fit preprocessing only on the training portion.
19. Train logistic regression, random forest, and gradient-boosted models.
20. Tune hyperparameters using validation performance.
21. Compare ROC-AUC, LogLoss, Brier score, and calibration.
22. Calibrate the strongest probability model if necessary.
23. Evaluate candidate approval thresholds under the economic model.
24. Measure default rate, expected profit, and thin-file inclusion gap.
25. Select the policy that maximizes the challenge objective subject to the risk constraint.
26. Retrain using permitted training data and generate test predictions without accessing hidden labels.

16. Recommended Experimental Design

- Experiment A — Baseline: logistic regression with core static features.
- Experiment B — Relational features: logistic regression with aggregated historical features.
- Experiment C — Nonlinear modeling: random forest.
- Experiment D — Boosting: gradient-boosted trees with tuned hyperparameters.
- Experiment E — Calibration: compare raw and calibrated probabilities.
- Experiment F — Decision policy: compare global thresholding with segment-aware policies.

Ablation studies should identify which feature families drive improvements, including static applicant information, bureau history, previous applications, person-level information, and tax-registry features.

17. Limitations

- The dataset is synthetic and should not be interpreted as representative of real American Express customers.
- The supplied economic values are challenge assumptions rather than actual lending economics.

- The thin-file inclusion gap is a project-specific metric and is not a complete legal or ethical fairness assessment.
- Temporal drift can make validation sensitive to the chosen cutoff period.
- High-dimensional relational aggregation can increase computational cost and overfitting risk.
- Model explanations do not automatically establish causal relationships.

18. Recommended Final System

The recommended final system is a calibrated gradient-boosted-tree probability model supported by a logistic-regression benchmark and evaluated against the random-forest alternative. The model should operate on applicant-level relational features, use an out-of-time validation framework, and produce well-calibrated default probabilities. A separate policy layer should determine approvals using the economic objective and portfolio default-rate ceiling.

For inclusivity, the system should explicitly monitor thin-file performance and compare the inclusion gap under the selected policy with the naïve single-threshold baseline. If a segment-aware policy substantially improves inclusion while remaining within the risk constraint and preserving strong profitability, it provides a stronger second-look lending strategy.

19. Conclusion

The Second-Look Lending problem is best approached as a combination of predictive modeling and decision optimization. Strong classification performance is necessary but insufficient: the final system must produce useful probabilities, translate them into economically meaningful decisions, and explicitly evaluate the treatment of thin-file applicants. A temporally validated, calibrated gradient-boosting model paired with a transparent policy layer provides a strong technical direction, while logistic regression and random forests offer important baselines and diagnostic comparisons.

References

27. American Express. Second-Look Lending Challenge Overview. Break Through Tech AI Studio, Fall 2026.
28. Break Through Tech AI Studio. SecondLook Dataset Guide.
29. Consumer Financial Protection Bureau. Circular 2022-03: Adverse Action Notification Requirements in Connection with Credit Decisions Based on Complex Algorithms.
30. Consumer Financial Protection Bureau. Regulation B, 12 CFR §1002.9.
31. scikit-learn documentation. Probability Calibration and Calibration Curves.
32. LightGBM documentation. Parameters.